

Q1. What is an I/O Module? Explain its functions.

Answer:

An **Input/Output (I/O) Module** acts as an interface between the **CPU/memory** and **external devices**.

♦ **Functions of I/O Module:**

1. **Control and Timing**
 - Coordinates data transfer between CPU and devices.
 - Maintains synchronization.
2. **Processor Communication**
 - Receives commands from CPU
 - Sends status signals
 - Transfers data
3. **Device Communication**
 - Interacts with peripherals
 - Converts signals into device format
4. **Data Buffering**
 - Temporarily stores data
 - Handles speed mismatch
5. **Error Detection**
 - Detects errors in transfer
 - Reports to CPU

Conclusion:

I/O module improves system efficiency by reducing CPU workload.

Q2. Explain Programmed I/O with advantages and disadvantages.

Answer:

Programmed I/O (PIO) is a method where CPU directly controls I/O operations.

♦ **Working:**

1. CPU sends command
2. CPU checks device status (polling)
3. Device becomes ready
4. Data transfer occurs

♦ **Features:**

- CPU fully involved
- Uses polling (busy waiting)
- No interrupts

♦ **Advantages:**

- Simple design
- No extra hardware
- Easy to implement

♦ **Disadvantages:**

- Wastes CPU time
- Inefficient
- Poor performance for large data

Example:

Keyboard input system

Q3. Explain Interrupt Driven I/O.

Answer:

Interrupt-driven I/O allows devices to **notify CPU only when ready**, instead of continuous checking.

♦ **Working:**

1. CPU sends request
2. CPU continues other work
3. Device sends interrupt
4. CPU executes ISR (Interrupt Service Routine)
5. Data transfer happens
6. CPU resumes work

♦ **Advantages:**

- Efficient CPU usage
- Faster response
- Better performance

♦ **Disadvantages:**

- Complex design
- Interrupt overhead
- Priority handling required

Q4. Explain DMA (Direct Memory Access).

Answer:

DMA allows devices to **transfer data directly to memory without CPU involvement.**

♦ **Working:**

1. CPU initializes DMA
2. DMA controller transfers data
3. CPU continues execution
4. DMA sends interrupt after completion

♦ **Types:**

- Memory → Memory
- Peripheral → Memory
- Memory → Peripheral

♦ **Advantages:**

- High speed
- Reduces CPU load
- Efficient for large data

Q5. Difference between Strobe and Handshaking.

Feature	Strobe Control	Handshaking
Number of Signals	Uses one signal (strobe)	Uses two signals (Data Valid & Acknowledge)
Communication Type	One-way	Two-way
Feedback Mechanism	No feedback	Feedback available
Reliability	Low	High
Synchronization	Poor synchronization	Proper synchronization
Error Handling	No error checking	Can detect errors
Speed	Faster (less control overhead)	Slightly slower (due to acknowledgment)

Complexity	Simple design	More complex
Data Loss Possibility	High	Very low
Control over Transfer	Sender assumes receiver is ready	Transfer happens only after confirmation
Hardware Requirement	Minimal	More hardware required
Suitability	Small/simple systems	Large/reliable systems

Q6. Explain Memory Hierarchy.

Answer:

Memory hierarchy organizes memory based on speed, cost, and size.

♦ Levels:

1. Registers (fastest)
2. Cache
3. Main Memory
4. Secondary Memory

♦ Key Concepts:

- Hit → Data found
- Miss → Data not found
- Hit Rate = Hits / Total Accesses
- Miss Rate = 1 - Hit Rate

♦ Locality:

- Temporal
- Spatial
- Sequential

Q7. Explain Cache Memory and Mapping Techniques

Cache memory is a small, high-speed memory located between the CPU and main memory. Its main purpose is to reduce memory access time by storing frequently used data and instructions.

it works on the principle of locality of reference:

- Temporal locality
- Spatial locality

Why Cache Memory is Needed

- Main memory is slow compared to CPU
- Cache bridges speed gap
- Improves system performance

Feature	Direct Mapping	Fully Associative Mapping	Set Associative Mapping
Basic Idea	Each memory block maps to one fixed cache block	Memory block can go anywhere in cache	Memory block maps to a set , not a single block
Mapping Function	Block $\rightarrow$ (Block mod N)	No fixed mapping	Block $\rightarrow$ Set (using mod)
Placement Flexibility	 No flexibility	 Full flexibility	 Partial flexibility
Search Method	Single location checked	Entire cache searched	Only set is searched
Address Fields	Tag + Index + Offset	Tag + Offset	Tag + Set + Offset
Speed	Fastest	Slowest	Medium
Hardware Complexity	Simple	Very complex	Moderate
Cost	Low	High	Medium
Cache Utilization	Poor	Best	Good

Conflict Misses	High	None	Reduced
Replacement Policy Needed	✗ Not required	✓ Required	✓ Required
Implementation Difficulty	Easy	Difficult	Moderate
Performance	Lower (due to conflicts)	Best performance	Balanced performance
Example Behavior	Same blocks compete for same location	No restriction	Limited competition within set

Q8 Numerical

Cache access time = 10ns
 Main memory access time = 200ns
 Hit rate = 99%

Find Effective Access Time (EAT)

Solution:

$$EAT = H \times T_c + (1-H) \times T_m$$

Substitute values:

$$\begin{aligned}
 EAT &= 0.99 \times 10 + 0.01 \times 200 \\
 EAT &= 9.9 + 2 \\
 \mathbf{EAT} &= \mathbf{11 \text{ ns}}
 \end{aligned}$$

Q9. Explain Virtual Memory

Virtual Memory is a memory management technique in which a part of secondary storage (disk) is used as an extension of main memory (RAM). It allows a system to run large programs even if RAM is limited.

- ◆ Need for Virtual Memory
 - Programs are larger than physical memory

- Multiple processes run simultaneously
- Efficient memory utilization is required
- Provides an illusion of large memory.

◆ Basic Concept

- CPU generates a logical (virtual) address
- This address is converted into a physical address
- Conversion is done using a page table

◆ Key Components

1. Logical Address

- Generated by CPU
- Also called virtual address

2. Physical Address - Actual location in RAM

3. Page Table - Stores mapping of virtual pages to physical frames

4. Page - Fixed-size block of virtual memory

5. Frame - Fixed-size block of physical memory

◆ Advantages

Allows execution of large programs
Efficient memory utilization
Supports multiprogramming
Reduces memory wastage

◆ Disadvantages

Slow if too many page faults
Disk access is very slow
Complex implementation

Q10. Explain Functions of Processor

The processor (CPU) is the brain of the computer system. It is responsible for executing instructions, performing calculations, and controlling all operations of the system. It performs a sequence of operations known as the instruction cycle.

◆ Basic Functions of Processor

The processor performs the following five major functions:

◆ 1. Fetch Instruction

- The processor reads an instruction from memory.
- The address of the instruction is stored in the Program Counter (PC).
- The instruction is transferred to the Instruction Register (IR).
- This step ensures the CPU knows what to execute next.

◆ 2. Decode Instruction

- The fetched instruction is interpreted (decoded).
- The control unit identifies:
 - Operation to perform
 - Operands involved

Example: ADD, SUB, LOAD, STORE

◆ 3. Fetch Data

- If instruction requires data, CPU fetches it from:
 - Memory
 - Registers
 - I/O devices

◆ 4. Execute Instruction

- The ALU (Arithmetic Logic Unit) performs:
 - Arithmetic operations (Addition, Subtraction)
 - Logical operations (AND, OR, NOT)

◆ 5. Write Back (Store Result)

- Result is stored in:
 - Register OR
 - Main memory

Q11. Explain Registers in Processor

Registers are small, high-speed storage locations inside the CPU used to store data, instructions, and addresses temporarily during execution.

They are the fastest memory in the computer system.

- ◆ Importance of Registers

- Reduce memory access time
- Improve performance
- Enable fast processing

- ◆ Types of Registers

- ◆ 1. User-Visible Registers - These are accessible by programmers.

- a) General Purpose Registers

- Used for multiple purposes
 - Can store data or addresses

- Example: AX, BX

- b) Data Registers

- Store actual data values
 - Used in arithmetic operations

- c) Address Registers

- Store memory addresses

Types:

- Index register
 - Stack pointer
 - Segment register

- d) Condition Code Registers (Flags)

These store the result status of operations:

- Zero Flag (Z) → Result is zero
 - Carry Flag (C) → Carry generated
 - Sign Flag (S) → Negative result
 - Overflow Flag (O) → Overflow occurs
 -

- ◆ 2. Control and Status Registers

These are used by the control unit.

- ◆ a) Program Counter (PC)

- ♦ b) Instruction Register (IR)
- ♦ c) Memory Address Register (MAR)
- ♦ d) Memory Buffer Register (MBR)

Q12. Explain Pipelining.

Pipelining allows parallel execution of instructions.

- ♦ Stages:
 1. Fetch
 2. Decode
 3. Execute
 4. Write
- ♦ Advantages:
 - Increases speed
 - Improves throughput
- ♦ Disadvantages:
 - Branch problem
 - Pipeline stall

Q13. Explain Structure of I/O Module.

The structure of an I/O module consists of multiple components that help in communication between CPU and devices.

- ♦ Components:
 - Data Register → Stores actual data
 - Status Register → Shows device condition (ready/busy)
 - Control Register → Holds command instructions
 - Address Decoder → Selects specific device
 - Bus Interface → Connects to system bus

Q14. What is Polling? Why is it inefficient?

Polling is a method where CPU continuously checks device status.

- ♦ Reason for inefficiency:
 - CPU remains busy (busy waiting)

- No parallel processing
- Wastes processing time

Q15. Difference between SRAM and DRAM

Feature	SRAM	DRAM
Speed	Fast	Slow
Cost	Expensive	Cheap
Refresh	Not needed	Required
Use	Cache	Main memory

Q16. Define Hit, Miss, Hit Ratio, Miss Penalty

- Hit → Data found in cache
- Miss → Data not found
- Hit Ratio (H) → Hits / Total accesses
- Miss Penalty → Time to fetch from main memory

Q17. What is Locality of Reference?

Programs tend to access data in predictable patterns:

- Temporal Locality → Recently used data reused
- Spatial Locality → Nearby data accessed
- Sequential Locality → Instructions executed in order

Q18. Paging vs Segmentation

Feature	Paging	Segmentation
Basic Concept	Divides memory into fixed-size pages	Divides memory into logical segments
Size of Block	Fixed	Variable
Basis of Division	Physical memory organization	Logical program structure
Fragmentation	Internal fragmentation	External fragmentation
Memory Allocation	Non-contiguous	May require contiguous allocation
Address Structure	Page number + Offset	Segment number + Offset
Mapping Table	Page Table	Segment Table
User Visibility	Invisible to programmer	Visible to programmer
Flexibility	Less flexible	More flexible
Protection & Sharing	Difficult	Easier (segment-wise)
Implementation	Simple	Complex

